

CO3 AT3

NAME: A K Selva Prabhu

REG NO: 192421224

COURSE CODE: CSA0904

COURSE NAME: PROGRAMMING FOR JAVA

1

```
class BankAccount {

    int balance = 1000;

    synchronized void deposit(int amount) {
        balance = balance + amount;
        System.out.println("Deposited: " + amount);
        System.out.println("Balance: " + balance);
    }

    synchronized void withdraw(int amount) {
        if (balance >= amount) {
            balance = balance - amount;
            System.out.println("Withdrawn: " + amount);
            System.out.println("Balance: " + balance);
        } else {
            System.out.println("Not enough balance");
        }
    }
}
```

```
static class Deposit extends Thread {
```

```
    BankAccount b;
```

```
    Deposit(BankAccount b) {
```

```
        this.b = b;
```

```
    }
```

```
    public void run() {
```

```
        b.deposit(500);
```

```
    }
```

```
}
```

```
static class Withdraw extends Thread {
```

```
    BankAccount b;
```

```
    Withdraw(BankAccount b) {
```

```
        this.b = b;
```

```
    }
```

```
    public void run() {
```

```
        b.withdraw(300);
```

```
    }
```

```
}
```

```
public static void main(String args[]) {
```

```
    BankAccount b = new BankAccount();
```

```
Deposit d = new Deposit(b);
```

```
Withdraw w = new Withdraw(b);
```

```
d.start();
```

```
w.start();
```

```
}
```

```
}
```

The screenshot shows the SIMATS Online Programming Lab interface. The header includes the SIMATS ENGINEERING logo, the lab name, and the user's name (SELVA PRABHU A K) with a dropdown arrow. On the left, a sidebar shows a question card for 'Q4 Product Price Input' with a green checkmark and '25 marks'. The main area is divided into three sections: 'Your Code', 'Input', and 'Output'. The 'Your Code' section contains a Java program for a BankAccount class. The 'Input' section shows 'stdin input'. The 'Output' section shows the program's execution results.

```
1 class BankAccount {
2
3     int balance = 1000;
4
5     synchronized void deposit(int amount) {
6         balance = balance + amount;
7         System.out.println("Deposited: " + amount);
8         System.out.println("Balance: " + balance);
9     }
10
11    synchronized void withdraw(int amount) {
12        if (balance >= amount) {
13            balance = balance - amount;
14            System.out.println("Withdrawn: " + amount);
15            System.out.println("Balance: " + balance);
16        } else {
17            System.out.println("Not enough balance");
18        }
19    }
20
21    static class Deposit extends Thread {
22        BankAccount b;
23
24        Deposit(BankAccount b) {
25            this.b = b;
26        }
27    }
28 }
```

Input: stdin input

Output: Deposited: 500
Balance: 1500
Withdrawn: 300
Balance: 1200

2

```
import java.util.Scanner;
```

```
class Login {
```

```
public static void main(String args[]) {
```

```
Scanner sc = new Scanner(System.in);
```

```
try {
```

```
    System.out.print("Username: ");
```

```
    String user = sc.nextLine();
```

```
System.out.print("Password: ");

int pass = sc.nextInt();

if (!user.equals("admin"))
    throw new IllegalArgumentException("Wrong username");

if (pass != 1234)
    throw new IllegalArgumentException("Wrong password");

System.out.println("Login Successful");

} catch (NumberFormatException e) {
    System.out.println("Invalid input");
} catch (IllegalArgumentException e) {
    System.out.println(e.getMessage());
} finally {
    System.out.println("Login process completed");
}
}
```



SIMATS Online Programming Lab
 JAVA PROGRAMMING

SELVA PRABHU A K
 192421224RA

QUESTIONS

Q1
 Bank Account ◊ Concurrent
 Deposit and Withdrawal
 25 marks

Q2
 Online Login System ◊ Multiple
 Exceptions
 25 marks

Q3
 Bank Withdrawal ◊ Insufficient
 Balance Exception
 25 marks

Q4
 Product Price Input ◊
 NumberFormatException
 25 marks

4/4 answered

Q2 Online Login System ◊ Multiple Exceptions

25 marks

Problem Statement:
Develop a Java program to simulate an online login system. The program should validate a username and password and perform operations that may generate different exceptions.

Requirements:
Accept login details.
Use try-catch-finally.
Handle at least two different built-in exceptions.
Display a message indicating whether the login operation was successful.
Ensure the finally block executes.

Your Code

```

1 import java.util.Scanner;
2
3 class Login {
4
5     public static void main(String args[]) {
6
7         Scanner sc = new Scanner(System.in);
8
9         try {
10             System.out.print("Username: ");
11             String user = sc.nextLine();

```

Input

admin

1234

Output

Username: Password: Login Successful

Login process completed

3

```
import java.util.Scanner;
```

```
class InsufficientBalanceException extends Exception {
```

```

    InsufficientBalanceException(String msg) {
        super(msg);
    }

```

```
public static void main(String args[]) {
```

```
    Scanner sc = new Scanner(System.in);
```

```
    System.out.print("Enter balance: ");
```

```
    int balance = sc.nextInt();
```

```
    System.out.print("Enter withdrawal: ");
```

```
    int amount = sc.nextInt();
```

```

try{

    if (amount > balance)

        throw new InsufficientBalanceException("Insufficient Balance");


    balance = balance - amount;


    System.out.println("Withdrawal Successful");

    System.out.println("Balance: " + balance);

} catch (InsufficientBalanceException e) {

    System.out.println(e.getMessage());

}

}

}

```

The screenshot displays the SIMATS Online Programming Lab interface. The header includes the SIMATS Engineering logo, the lab name, and the user's name (SELVA PRABHU A K) with a dropdown arrow. The main content area is titled "Q3 Bank Withdrawal ♦ Insufficient Balance Exception" with a value of 25 marks. It contains a "Problem Statement" section describing the task of simulating a bank withdrawal with an exception, followed by "Requirements" such as reading account balance, creating a user-defined exception, and displaying the updated balance. Below this is a "Your Code" section with a text editor containing Java code for the `InsufficientBalanceException` class and its `main` method. To the right of the code is an "Input" section with two text boxes containing the values "10000" and "3000". Below the input is an "Output" section showing the program's execution results: "Enter balance: Enter withdrawal: Withdrawal Successful Balance: 7000". On the left side, there is a "QUESTIONS" sidebar listing four questions (Q1 to Q4) with their respective topics and marks, and a "4/4 answered" status at the bottom.

4

```
import java.util.Scanner;
```

```
class Product {
```

```

public static void main(String args[]) {

    Scanner sc = new Scanner(System.in);

    System.out.print("Enter price: ");

    String price = sc.nextLine();

    try{

        double p = Double.parseDouble(price);

        System.out.println("Price: " + p);

    } catch (NumberFormatException e) {

        System.out.println("Invalid price");

    }

}

}

```


SIMATS Online Programming Lab
JAVA PROGRAMMING

 SELVA PRABHU A K
192421224RA

QUESTIONS

- Q1** Bank Account ◊ Concurrent Deposit and Withdrawal 25 marks
- Q2** Online Login System ◊ Multiple Exceptions 25 marks
- Q3** Bank Withdrawal ◊ Insufficient Balance Exception 25 marks
- Q4** Product Price Input ◊ NumberFormatException 25 marks

4/4 answered

Q4 Product Price Input ◊ NumberFormatException 25 marks

Problem Statement:
 Develop a Java program that accepts the price of a product as a String and converts it into a numeric value. Handle invalid numeric input without terminating the program.

Requirements:
 Read product price as a String.
 Convert it using Integer.parseInt() or Double.parseDouble().
 Handle NumberFormatException.
 Display an appropriate error message.

Your Code

```

1 import java.util.Scanner;
2
3 class Product {
4
5     public static void main(String args[]) {
6
7         Scanner sc = new Scanner(System.in);
8
9         String price = sc.nextLine();
10
11

```

Input

abc

Output

Invalid price

